

Phase 7 – ConfigMaps & Secrets in Amazon EKS

Objective

In this phase, we will learn how Kubernetes applications receive configuration values and sensitive credentials without hardcoding them into the application code or Docker image.

By the end of this phase, we will understand:

- ConfigMaps
- Secrets
- Environment Variable Injection
- Volume Mounts
- How Kubernetes stores these objects
- How Pods consume them

Why Do We Need ConfigMaps and Secrets?

Imagine an application requiring:

APP_ENV=production

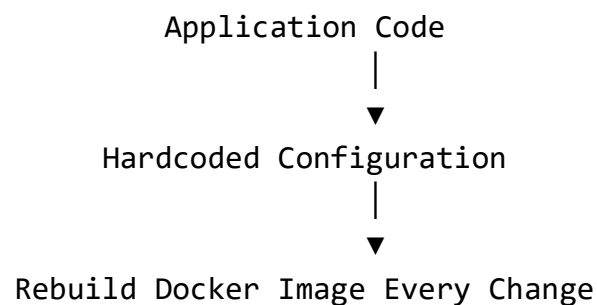
DB_HOST=mysql-prod

DB_PORT=3306

DB_USERNAME=admin

DB_PASSWORD=SuperSecret123

Without ConfigMaps and Secrets:



Problems:

- Poor security
- Difficult maintenance
- Image rebuild required for every change
- Credentials exposed in source code

Kubernetes Solution

Kubernetes separates:

Application Code

from

Application Configuration

and

Application Credentials

using:

ConfigMap

Stores: Non-Sensitive Data

Examples:

APP_ENV

DB_HOST

DB_PORT

APP_VERSION

Secret

Stores: Sensitive Data

Examples:

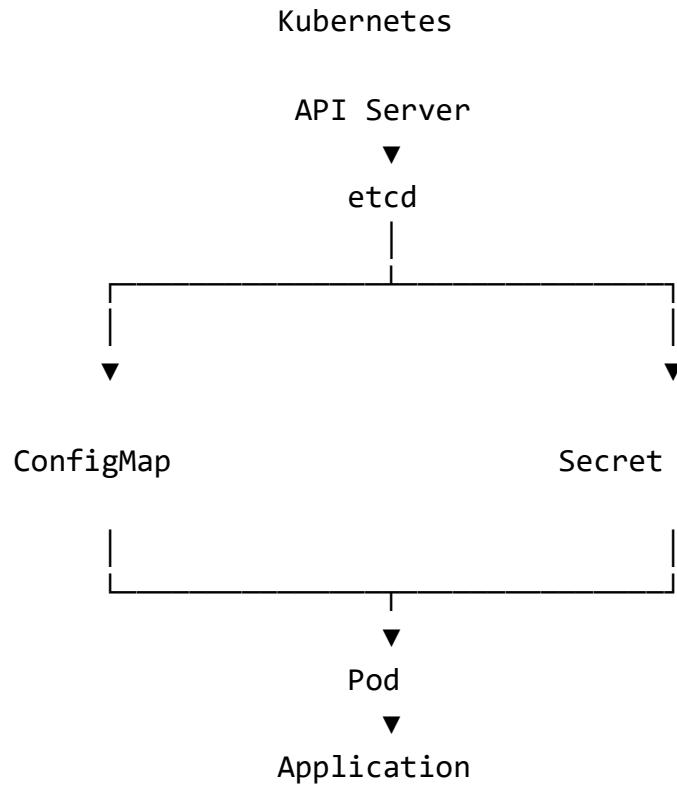
Passwords

Tokens

API Keys

Certificates

Architecture



Step 1 – Create ConfigMap

Create file:

```
apiVersion: v1
kind: ConfigMap
```

```
metadata:
  name: app-config
```

```
data:
  APP_ENV: production
  APP_NAME: nginx-app
  DB_HOST: mysql-prod
  DB_PORT: "3306"
```

Save as:

configmap.yaml

Apply:

```
kubectl apply -f configmap.yaml
```

Expected Output:

configmap/app-config created

Verify ConfigMap

```
kubectl get configmap
```

Output:

NAME	DATA	AGE
app-config	4	7s
kube-root-ca.crt	1	2d

Describe ConfigMap

```
kubectl describe configmap app-config
```

Output:

```
APP_ENV=production
APP_NAME=nginx-app
DB_HOST=mysql-prod
DB_PORT=3306
```

What Happened Behind The Scenes?

```
kubectl
```



API Server



etcd



ConfigMap Stored

Important:

No Pod Created

No Deployment Created

No Container Started

ConfigMap is only a Kubernetes object stored in etcd.

Step 2 – Consume ConfigMap as Environment Variables

Create:

```
apiVersion: v1
kind: Pod

metadata:
  name: configmap-demo

spec:
  containers:
    - name: demo
      image: busybox

      command:
        - sleep
        - "3600"

      env:
        - name: APP_ENV
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_ENV

        - name: APP_NAME
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: APP_NAME

        - name: DB_HOST
          valueFrom:
            configMapKeyRef:
              name: app-config
              key: DB_HOST

        - name: DB_PORT
          valueFrom:
            configMapKeyRef:
```

```
name: app-config
key: DB_PORT
```

Save as:

configmap-pod.yaml

Apply: `kubectl apply -f configmap-pod.yaml`

Verify: `kubectl get pods`

Output: configmap-demo Running

Verify ConfigMap Injection

`kubectl exec -it configmap-demo -- env`

Output:

```
APP_ENV=production
APP_NAME=nginx-app
DB_HOST=mysql-prod
DB_PORT=3306
```

Step 3 – Create Secret

Create:

```
apiVersion: v1
kind: Secret
```

```
metadata:
  name: db-secret
```

```
type: Opaque
```

```
stringData:
  DB_USERNAME: admin
  DB_PASSWORD: SuperSecret123
```

Save as: secret.yaml

Apply: `kubectl apply -f secret.yaml`

Expected: secret/db-secret created

Verify Secret

```
kubectl get secret
```

Output:

NAME	TYPE	DATA
db-secret	Opaque	2

Describe Secret: `kubectl describe secret db-secret`

Output:

DB_PASSWORD: 14 bytes

DB_USERNAME: 5 bytes

Notice: Actual Values Hidden

What Is Opaque?

type: Opaque

means: Generic Kubernetes Secret

Used for:

Passwords

API Keys

Tokens

Credentials

Step 4 – Consume ConfigMap + Secret as Environment Variables

Create:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: config-secret-demo
```

```
spec:
```

```
  containers:
```

```
    - name: demo
```

```
      image: busybox
```

```
      command:
```

```
        - sleep
```

- "3600"

env:

- name: APP_ENV
valueFrom:
 configMapKeyRef:
 name: app-config
 key: APP_ENV
- name: APP_NAME
valueFrom:
 configMapKeyRef:
 name: app-config
 key: APP_NAME
- name: DB_HOST
valueFrom:
 configMapKeyRef:
 name: app-config
 key: DB_HOST
- name: DB_PORT
valueFrom:
 configMapKeyRef:
 name: app-config
 key: DB_PORT
- name: DB_USERNAME
valueFrom:
 secretKeyRef:
 name: db-secret
 key: DB_USERNAME
- name: DB_PASSWORD
valueFrom:
 secretKeyRef:
 name: db-secret
 key: DB_PASSWORD

Apply: `kubectl apply -f config-secret-pod.yaml`

Verify: `kubectl exec -it config-secret-demo -- env`

Output:


```
APP_ENV=production
APP_NAME=nginx-app
DB_HOST=mysql-prod
DB_PORT=3306
```

```
DB_USERNAME=admin
DB_PASSWORD=SuperSecret123
```

Step 5 – Consume ConfigMap and Secret as Mounted Volumes

Create:

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: volume-demo
```

```
spec:
  containers:
    - name: demo
      image: busybox

      command:
        - sleep
        - "3600"

      volumeMounts:
        - name: config-volume
          mountPath: /etc/config

        - name: secret-volume
          mountPath: /etc/secret
```

```
volumes:

  - name: config-volume
    configMap:
      name: app-config

  - name: secret-volume
```

```
secret:
  secretName: db-secret
```

Save as: volume-demo.yaml

Apply:

```
kubectl apply -f volume-demo.yaml
```

Verify Mounted Files

Enter Pod: `kubectl exec -it volume-demo -- sh`

ConfigMap Files

```
ls /etc/config
```

Output:

```
APP_ENV
APP_NAME
DB_HOST
DB_PORT
```

Read Values: `cat /etc/config/APP_ENV`

Output: production

```
cat /etc/config/DB_HOST
```

Output: mysql-prod

Secret Files

```
ls /etc/secret
```

Output:

```
DB_PASSWORD
DB_USERNAME
```

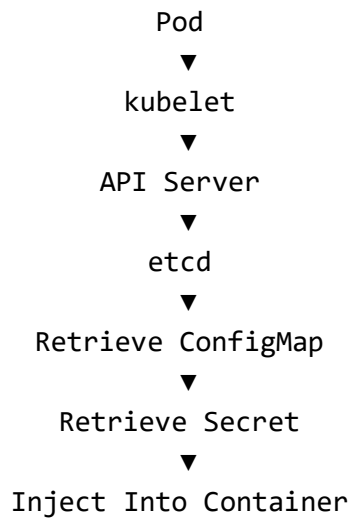
Read Values: `cat /etc/secret/DB_USERNAME`

Output: admin

```
cat /etc/secret/DB_PASSWORD
```

Output: SuperSecret123

Internal Working: When Pod Starts:



Environment Variables vs Mounted Volumes

Method

Environment Variables

Mounted Files

Examples:

Nginx Configurations

TLS Certificates

Java Property Files

SSH Keys

typically use mounted volumes.

Usage

Simple application settings

Configuration files, certificates, keys

Interview Questions

What is a ConfigMap?

A ConfigMap stores non-sensitive application configuration outside the container image.

What is a Secret?

A Secret stores sensitive information such as passwords, API keys, and tokens.

Where are ConfigMaps and Secrets stored?

Inside Kubernetes etcd through the API Server.

Can Pods access etcd directly?

No.

Pods receive values through Kubernetes APIs and kubelet.

How can Pods consume ConfigMaps and Secrets?

- Environment Variables
- Mounted Volumes

Keep these resources for future reference and experimentation before proceeding to Phase 8 (Persistent Volumes and Storage)